The Syntax of Propositional Logic

Definition A ***propositional formula*** is either

- an ***atomic formula***, typically an uppercase variable P, Q, R etc.; or
- a ***compound formula*** build up using connectives.

Two special atomic formulas:

- ***true*** (the proposition which is always true)
- ***false*** (the proposition which is always false)

In summary: If p and q are propositional formulas, then so are the following:

true	:		:	*truth*
false	:		:	*falsity*
$\mathrm{P}, \mathrm{Q}, \ldots$	:		:	*propositional variables*
$\neg\,\mathrm{p}$	:	*not* p	:	*negation*
$\mathrm{p} \wedge \mathrm{q}$	:	p *and* q	:	*conjunction*
$\mathrm{p} \vee \mathrm{q}$	:	p *or* q	:	*disjunction*
$\mathrm{p} \Rightarrow \mathrm{q}$	:	p *implies* q	:	*implication*
$\mathrm{p} \Leftrightarrow \mathrm{q}$	:	p *if, and only if,* q	:	*equivalence*

Note: p and q (lower case) are ***not*** propositional variables; they are *metavariables* which stand for arbitrary formulas.

Parentheses and Precedences

We use parentheses to disambiguate expressions.

For example, $P \vee Q \Rightarrow R$ could be read in two very different ways:

$$(P \vee Q) \Rightarrow R \qquad \text{or} \qquad P \vee (Q \Rightarrow R)$$

We also define a precedence for connectives to reduce the need for parentheses (and thus increased readability without creating ambiguity):

- $\neg$ binds more tightly than $\wedge$
- $\wedge$ binds more tightly than $\vee$
- $\vee$ binds more tightly than $\Rightarrow$
- $\Rightarrow$ binds more tightly than $\Leftrightarrow$

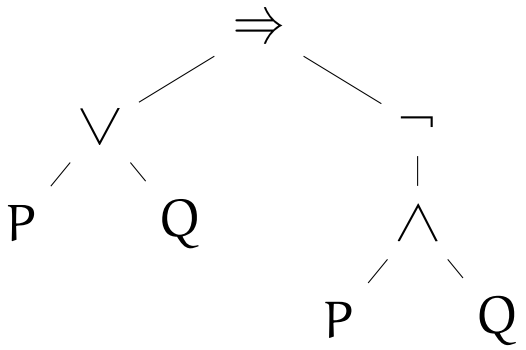
Apart from this, operations are applied right-to-left.

Example: $P \Rightarrow \neg Q \vee R \Rightarrow Q$ reads as $P \Rightarrow \Big(\big((\neg Q) \vee R\big) \Rightarrow Q\Big)$

Syntax Trees

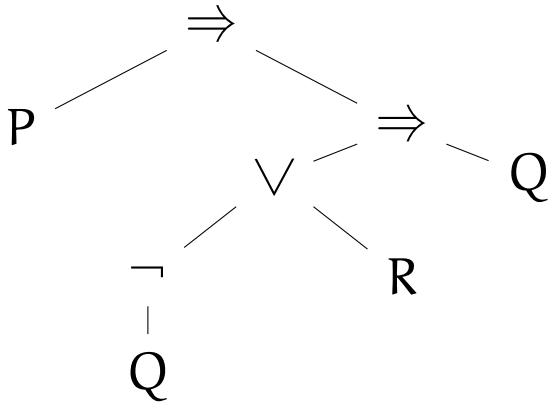
Syntax trees provide a way to visualise the structure of propositional formulas.

The formula $(P \vee Q) \Rightarrow \neg(P \wedge Q)$ is depicted by the syntax tree to the right.



The syntax tree makes parsing clear without the need for parentheses or precedence rules.

The formula $P \Rightarrow \neg Q \vee R \Rightarrow Q$ is depicted by the syntax tree to the right.



The syntax tree also helps to evaluate this expression based on truth values for variables P, Q, R, and to determine how to place brackets.

Well-formed Formulas (wff)

A syntactically-correct formula is called a ***well-formed formula (wff)***.

For example, the string of symbols

$$\neg(\mathrm{P} \vee (\wedge(\mathrm{Q}, \mathrm{R}) \rightarrow \mathrm{P})))$$

fails to be a well-formed formula for many reasons:

- the conjunction $\wedge$ is not between two subformulas.
- The symbols , and $\rightarrow$ are not connectives of propositional logic.
- the parentheses do not match (there is one too many closing parentheses).

We would not be able to form this string by applying the formation rules for formulas; and we wouldn't be able to draw a syntax tree for it.

Well-formed Formulas and Syntax Trees

Exercise: Which of the following are well-formed formulas?

Rewrite each well-formed formula using a minimal number of parentheses, and draw its syntax tree.

1. $((P \rightarrow Q) \rightarrow (Q \rightarrow P))$

2. $P \vee Q(\wedge P)$

3. $(P \vee Q) \wedge P$

4. $(P \vee Q) \Leftrightarrow (R \neg S))$

Defining Further Propositional Operators

The *exclusive or* operation $\mathrm{p} \oplus \mathrm{q}$ can be defined by saying that either p is true and q is false, or p is false and q is true:

$$\mathrm{p} \oplus \mathrm{q} \qquad \Leftrightarrow \qquad (\mathrm{p} \wedge \neg \mathrm{q}) \vee (\neg \mathrm{p} \wedge \mathrm{q})$$

Alternatively, $\mathrm{p} \oplus \mathrm{q}$ says that one of p and q is true if, and only if, the other one is *not* true:

$$\mathrm{p} \oplus \mathrm{q} \qquad \Leftrightarrow \qquad \mathrm{p} \Leftrightarrow \neg \mathrm{q}$$

Exercise: Express the following connectives using the connectives of propositional logic.

1. The NAND operation $\mathrm{p} \mid \mathrm{q}$ which is true if, and only if, p and q are not both true.
2. The NOR operation $\mathrm{p} \downarrow \mathrm{q}$ which is true if, and only if, neither p nor q are true.

Modelling with Propositional Logic

Propositional Logic is important for modelling real-life scenarios, in which propositional variables represent particular properties which can be true or false.

Example: Consider the following lines of code:

```
if CabinPressure < MinPressure
     then PrepareForLanding;
if FlightHeight < MinHeight
     then PrepareForLanding;
```

A software engineer optimises this to

```
if (CabinPressure < MinPressure
          and FlightHeight < MinHeight)
     then PrepareForLanding;
```

Modelling with Propositional Logic

Using variables
- Pressure for `CabinPressure < MinPressure`
- Height for `FlightHeight < MinHeight`
- Land for `PrepareForLanding`

we can express the program as a propositional formula

$$(\text{Pressure} \Rightarrow \text{Land}) \ \wedge \ (\text{Height} \Rightarrow \text{Land})$$

while the suggested optimisation corresponds to

$$(\text{Pressure} \wedge \text{Height}) \ \Rightarrow \ \text{Land}$$

These two formulas are *not* equivalent. The correct variant, one which is equivalent to the formula corresponding to the original program, would be

$$(\text{Pressure} \vee \text{Height}) \Rightarrow \text{Land}$$

That is, the optimised code should be

```
if (CabinPressure < MinPressure
        or FlightHeight < MinHeight)
    then PrepareForLanding;
```